NYTimes verification stack — DoubleVerify + GeoEdge dissection (2026-10-31)

Subject: the ad-verification layer on `nytimes.com` — what it collects and how robust the bot-detection is.
Method: clean-launch real Chrome (DataDome requires it), captured the actual verification bundles from the CDN, static + runtime analysis. All telemetry observed or locally fulfilled; nothing forwarded.
Result: DoubleVerify runs a **real browser-detection engine** (explicitly detects Playwright/Puppeteer/Selenium + a dedicated PuppeteerExtraStealth rule), a broad viewability/human-signal collector, and GeoEdge runs a separate **malvertising scanner**. Robust against detectable automation — which is exactly why our clean-launch (no injection) passed.

---

1. The verification stack (what fired on NYT)

Vendor	Script	Role
DoubleVerify	`dvtp_src.js` (10 KB, injection loader) + `dv-measurements11166.js` (527 KB, the engine)	Viewability + IVT + brand-safety measurement
DoubleVerify	`tps.doubleverify.com/visit.js` + beacons to `tps-dn-ue1.doubleverify.com` / `maestro-ue1.tps.dv.tech`	measurement + `google_nid=doubleverify_ddp` cookie-sync
GeoEdge	`rumcdn.geoedge.be/grumi-ip.js` + `grumi.js` (1 MB)	malvertising/ad-security scanner
Scorecard/Comscore	`sb.scorecardresearch.com/beacon.js`	audience measurement

2. DoubleVerify's browser-detection engine (the robustness)

`dv-measurements11166.js` defines an enum of **detected browser types**:

```
Chrome, ChromeWebView, Firefox, IE, Opera, Safari, SafariWebView,
Selenium, Playwright, PuppeteerVanilla, PuppeteerExtraStealth, Unknown
```

Three rule types:
- **ruleType 1** — UA regex (real-browser vs webview detection)
- **ruleType 2** — JS property probe (automation detection)
- **ruleType 3** — composite (e.g., Safari-standalone check)

The automation rules (what they probe):

Detected	Check
Selenium	`documentElement` has `webdriver` / `fxdriver` / `__webdriver_unwrapped` / `__webdriver_script_fn` / `__webdriver_evaluate` / `__fxdriver_*` attributes; `window.webdriver` / `domAutomation` / `_WEBDRIVER_ELEM_CACHE`
Playwright	`window.__pwInitScripts` **or** `window.__playwright__binding__`
PuppeteerVanilla	`navigator.webdriver === true`

Detected	Check
PuppeteerExtraStealth	dedicated `getIsExtraStealthBrowser()` probe

Why our clean-launch passed: a manually-launched real Chrome has **none** of those markers — no `__pwInitScripts`, no `__playwright__binding__`, no `__webdriver_*` attributes, `navigator.webdriver=false`. The Playwright-launched runs (which DataDome blocked at the edge anyway) *would* have tripped DV's Playwright rule via `__pwInitScripts` — DV is genuinely robust against instrumented browsers.

3. What DV collects (the signal surface)

- **Viewability/human signals:** `mousemove`/`touchmove` listeners (on the ad window + top window), `click`/`mouseover`/`focus` event tracking per ad candidate, scroll position, visibility state (`prerender`/`visibilitychange`).
- **Browser surface:** navigator, userAgent, platform, screen (58 refs), canvas (5), hardwareConcurrency, deviceMemory, performance/timing (53 refs), referrer, cookies.
- **Frame integrity:** 198 `frame` + 47 `parent` refs — it walks the iframe chain (which window am I in, is the top accessible, framing/clickjacking checks).

The output feeds: **viewability** (was ≥50% visible ≥1s), **IVT classification** (invalid traffic via the browser-detection rules), and **brand-safety/context**.

4. GeoEdge (the other half — malvertising)

`grumi.js` (1 MB) is a different job: **ad-security**.
- Hooks ad iframes (`div.ad iframe`), `popupMonitoring.js` (tracks popups/redirects/`window.open`), and scans creative behavior.
- Carries a large **blocklist of known-bad ad destinations** (redirect chains, scam/unauthorized domains).
- Purpose: stop **malvertising** — malicious creatives that phish, drive-by download, or redirect users. Protects the *user*, complements DV's measurement.

5. How robust is it? (honest assessment)

Layer	Robust against	Not a defense against
DV browser-detection	Instrumented browsers (Playwright/Puppeteer/Selenium markers), webdriver flags	**Clean-launch real Chrome** (indistinguishable from human)
DV viewability	Scrolled-out-of-view / hidden slots	A real-browser session that keeps ads in viewport
GeoEdge	Malvertising creatives	Legitimate-looking ads from a clean session
DataDome (edge)	Automated launches	Clean-launch real Chrome (the same ladder as hCaptcha/BootBarn)

The honest conclusion: DV + GeoEdge are **genuinely well-built** — DV's explicit Playwright/PuppeteerExtraStealth rules show real sophistication (most

verification vendors don't enumerate stealth plugins). But they are **auditors, not gates**: they measure a session that already passed the edge. A clean-launch real Chrome is indistinguishable from a human at every one of their checks. The robustness is real, but the boundary is still the edge (DataDome), and even that is a "clean launch clears it" ladder.

6. Artifacts

File	What
`assets/dv-measurements11166.js`	DoubleVerify measurement engine (527 KB)
`assets/dvtp_src.js`	DV injection loader
`assets/grumi.js`	GeoEdge malvertising scanner (1 MB)
`assets/grumi-ip.js`	GeoEdge lightweight probe
`artifacts/dv-requests.json`	full verification-vendor request flow
`scripts/nyt-dv.mjs`	the capture probe

7. Summary line

> *"DoubleVerify runs a real bot-detection engine — it explicitly detects Playwright (`__pwInitScripts`), Puppeteer (webdriver flag), PuppeteerExtra Stealth (dedicated probe), and Selenium's many markers, plus viewability and behavioral signals; GeoEdge separately scans for malvertising. Robust against instrumented browsers — but a clean-launch real Chrome is indistinguishable from a human at every check. Verification is an auditor, not a gate."*